

Se utilizó LLM para producir la tabla final de resultados:

El prompt fue el siguiente:

A partir de los archivos entregados (en referencia a los csv y los workload) puedes generar una tabla resumen con el tiempo de las consultas representativas de cada consulta representativa definida la parte 1 (se entrega como texto la parte 1)

Respuesta:

El documento tiene la siguiente tabla lista para pegar en el world:

Una sola tabla con todas las clases del Paso 1 organizadas así:

- 6 columnas: Clase | Índice aplicado | Query | Baseline (ms) | Optimizado (ms) | Reducción
- Cada grupo de consultas tiene una fila de encabezado azul oscuro con el nombre de la clase y descripción
- 2 consultas representativas por clase, alternando color de fila
- Fila de totales al final con los 14.237 ms → 3.594 ms y el 74.8% de reducción
- Nota al pie con el speedup de 3.96

(Se entrega la tabla)

Se utilizó LLM para verificar los primeros pensamientos de posibles índices que teníamos:

Se utilizó LLM para verificar los primeros pensamientos de posibles índices que teníamos:

El prompt fue el siguiente:

A partir de los archivos entregados (workload.sql con 100 consultas, schema.sql con las tablas y load.sql para la carga de datos) se realizó un análisis previo. Necesito que confirmes si los siguientes índices podrían optimizar el tiempo de ejecución:

- HASH INDEX en el email de users (considerando el uso de LOWER)
- B+ Tree en ORDER_Date de la tabla orders
- HASH INDEX en BOOK_ID de reviews
- B+ Tree en TITLE de la tabla books

Además, confirmar si la siguiente agrupación de consultas es correcta:

- Búsquedas exactas por email en users (con y sin LOWER)
- Actualizaciones de stock en inventario mediante llaves primarias

- Consultas de reviews por book_id (exacta) y review_date
- Consultas anidadas con COUNT y book_id en reviews
- Filtros combinados en books (género exacto, rangos de precio y año)
- Búsquedas en books utilizando el parámetro LIKE
- Consultas en orders por rango de fecha y ordenamiento cronológico
- Joins triples (books, orders, order_items) con filtros de fecha y agrupaciones por título o género

Entrega una opinión objetiva sobre si lo planteado es correcto.

La respuesta:

Análisis del workload y evaluación de sus propuestas

Clases de consultas Identificadas (lo que realmente hay)

Antes de evaluar los índices, hay que confirmar cuántas clases distintas existen y qué hacen exactamente:

Clase	Ejemplos	Patrón
Q-users-exact	003,007,008,017-019,022,025,030...	<code>WHERE email = '...' (case-sensitive)</code>
Q-users-lower	032,038,065,073,079,089,097	<code>WHERE LOWER(email) = LOWER('...')</code>
Q-inventory-update	005,006,013,026,043,068,088...	<code>UPDATE inventory WHERE book_id = X AND warehouse_id = Y</code>
Q-reviews-book	004,009,011,016,020,021,024,028...	<code>WHERE book_id = X ORDER BY review_date DESC</code>
Q-reviews-count	058	<code>SELECT COUNT(*) FROM (GROUP BY book_id HAVING COUNT(*) >= 10 AND AVG(rating) > 4)</code>
Q-books-filter	001,040,041,070,072,096	<code>WHERE genre = X AND price BETWEEN A AND B AND publication_year >= Y</code>
Q-books-like	014,023,046,055,063,086,095...	<code>WHERE title LIKE 'Prefix%' ORDER BY title</code>
Q-orders-range	027,081,084,098	<code>WHERE order_date >= X AND order_date < Y ORDER BY order_date DESC</code>
Q-orders-revenue	002,010,012,015,029,042,059,066,082,091	<code>Triple JOIN + WHERE order_date BETWEEN + GROUP BY b.book_id, b.title</code>
Q-orders-genre	033,037,057	<code>Triple JOIN + WHERE order_date BETWEEN + GROUP BY b.genre, DATE_TRUNC(month)</code>
Q-orders-premium	056,061,094	<code>JOIN con users + WHERE is_premium = TRUE AND order_date BETWEEN</code>

Eso da 11 clases reales. Su agrupación en 8 agrupa correctamente algunos pero pierde matices importantes. Más detalles abajo.

Evaluación de los índices propuestos

✅ **HASH INDEX en `users.email`**

Correcto, pero Incompleto. Las consultas con `WHERE email = '...'` (Q-users-exact) son búsqueda exacta — un Hash Index es ideal aquí. Sin embargo hay que notar que `email` ya tiene un UNIQUE constraint en el schema, lo que en PostgreSQL crea automáticamente un B-Tree sobre esa columna. Es decir, ya existe un índice funcional. Agregar uno Hash explícito puede mejorar marginalmente las búsquedas exactas, pero el beneficio real es menor de lo esperado porque el B-Tree del UNIQUE ya funciona bien para igualdad.

El problema crítico que Identificaron correctamente: las consultas con `LOWER(email) = LOWER('...')` (Q-users-lower) no usan ningún índice existente porque aplican una función. La solución correcta es un índice funcional:

```
sql
CREATE INDEX ON users (LOWER(email));
```

Esto cubre ambas clases. Sin este índice, las consultas LOWER hacen full scan de users.

✅ **B-Tree INDEX en `orders.order_date`**

Correcto y bien justificado. Aparece en múltiples clases: Q-orders-range (búsqueda en rango directa), Q-orders-revenue y Q-orders-genre (el filtro `WHERE order_date BETWEEN` en el JOIN), y Q-orders-premium (idem). Un B-Tree es la estructura correcta para rangos. Este índice tiene alto impacto porque es la columna de filtro en al menos ~20 consultas.

✅ **HASH INDEX en `reviews.book_id`**

Parcialmente correcto. Las consultas de reviews siempre usan `WHERE book_id = X` (igualdad exacta), lo que favorece Hash. Sin embargo, hay un matiz importante: todas estas consultas también hacen `ORDER BY review_date DESC`. Si se crea un índice compuesto (`book_id, review_date DESC`) como B-Tree, el motor puede satisfacer tanto el filtro como el orden sin un sort adicional. Un Hash Index sobre solo `book_id` filtra bien pero obliga a un sort posterior. Para el patrón exacto que tiene el workload, el índice compuesto B-Tree es más poderoso que el Hash simple.

⚠️ **B-Tree en `books.title`**

Correcto solo para LIKE con prefijo fijo. Las consultas `WHERE title LIKE 'PrefixN'` sí pueden usar un B-Tree sobre title, porque el patrón ancla por la izquierda. Esto es válido en PostgreSQL con el operador class correcto (`trailing pattern_ops` si el collation no es C). El índice también sirve para el `ORDER BY title` — acompaña a cada query LIKE. Sin embargo, este índice no aporta nada para Q-books-filter (que filtra por genre/price/year, no).

Al final se cambiaron algunos índices luego de ver bien el tema del materialized. Ilm no funcionó tan bien porque algunas sugerencias estaban bien incompletas.